



INTERNSHIP REPORT

PCAP Preprocessing AI Agent

for Automotive Ethernet Testing

Preprocessing and preparation of Automotive Ethernet network traces for downstream analysis

Prepared by:	Balkis Hmidi
Host Organization:	KPIT Technologies Tunisia
Academic Year:	2025–2026

Automotive Engineering Internship

Acknowledgments

I would like to express my sincere gratitude to everyone who contributed to making this internship a valuable and enriching experience.

First and foremost, I would like to sincerely thank my internship supervisor, **Mr. Montassar Ben Arfia**, for his guidance, support, and supervision throughout the internship. His advice and feedback contributed significantly to the progress of this work and to my professional development during this experience.

I would also like to express my appreciation to **KPIT Technologies Tunisia** for giving me the opportunity to complete my internship within the company and to work on an interesting project in the field of Automotive Ethernet and network data analysis.

My sincere thanks also go to all the members of the team and everyone who contributed, directly or indirectly, to this internship experience. I am grateful for their support, collaboration, and the professional environment they provided throughout this period.

Balkis Hmidi

Abstract

The increasing adoption of Automotive Ethernet has led to the generation of large volumes of network communication data during vehicle testing and validation activities. Packet capture (PCAP) traces contain valuable information about communication behavior, protocols, and network events; however, their size and the presence of repetitive or low-value traffic can make their analysis time-consuming and difficult.

This internship project addresses the preparation of Automotive Ethernet PCAP traces for downstream automated and AI-based analysis. The objective is to transform raw communication data into a reduced and structured representation while preserving the information required to accurately understand relevant network behavior.

The proposed solution follows a modular preprocessing architecture organized around three main stages. The first stage extracts meaningful automotive communication events from the input data. The second stage performs noise filtering and context compression to reduce repetitive, redundant, or low-value information while preserving relevant communication context. The final stage aggregates the resulting information and generates a structured communication context intended for downstream AI analysis.

The system is implemented using Python-based network processing and an AI agent workflow that coordinates the available preprocessing operations. The architecture separates the core processing logic, specialized tools, and workflow orchestration to improve modularity and maintainability.

The resulting approach provides a reusable foundation for preparing Automotive Ethernet communication traces for further applications such as protocol analysis, automated validation, anomaly detection, and AI-assisted network investigation.

Keywords: Automotive Ethernet, PCAP, network preprocessing, AI agent, network traffic analysis, context compression, Automotive Ethernet testing.

Contents

Acknowledgments	i
Abstract	ii
List of Abbreviations	vii
General Introduction	1
1 Project Context	3
1.1 Host Organization Presentation	3
1.1.1 Company Overview	3
1.1.2 KPIT Tunisia and Organizational Context	3
1.1.3 Testing and Validation Context	4
1.2 Problem Statement	5
1.2.1 Automotive Ethernet Testing Challenges	5
1.2.2 Current Limitations	6
1.3 Proposed Solution	7
1.3.1 PCAP Preprocessing AI Agent	7
1.3.2 Expected Benefits	7
2 Technologies and Tools	9
2.1 Automotive Ethernet Technologies	9
2.1.1 Ethernet Fundamentals	9
2.1.2 Automotive Ethernet	10
2.1.3 Virtual Local Area Networks	10
2.1.4 TCP, IP, and UDP	10
2.2 Automotive Communication Protocols	11
2.2.1 SOME/IP	11
2.2.2 SOME/IP Service Discovery	12
2.2.3 Diagnostics over Internet Protocol	12
2.2.4 Address Resolution Protocol and Dynamic Host Configuration Protocol	12
2.3 Data Processing and AI Agent Technologies	13
2.3.1 Python	13
2.3.2 Scapy	13
2.3.3 LangChain	14

2.3.4	LangGraph	14
2.4	Development Environment	14
2.4.1	Visual Studio Code	14
2.4.2	Git	15
2.5	Chapter Summary	15
3	System Architecture	16
3.1	Requirements Analysis	16
3.1.1	Functional Requirements	16
3.1.2	Non-Functional Requirements	17
3.2	Global Architecture	18
3.2.1	Architectural Overview	18
3.2.2	Data Input Layer	19
3.2.3	Agent Orchestration Layer	19
3.3	Preprocessing Tools	20
3.3.1	Automotive Event Extraction	20
3.3.2	Noise Filtering and Context Compression	20
3.3.3	Context Builder and Report Generator	20
3.4	Data Processing Workflow	21
3.5	State and Information Flow	22
3.6	Chapter Summary	23
4	Implementation and Validation	24
4.1	Development Methodology	24
4.2	Project Implementation Structure	25
4.2.1	Separation of Responsibilities	25
4.2.2	Core Processing Layer	25
4.2.3	Tool Layer	25
4.2.4	Agent and Workflow Layer	26
4.3	Preprocessing Tool Implementation	26
4.3.1	Automotive Event Extraction	26
4.3.2	Noise Filtering and Context Compression	27
4.3.3	Context Builder and Report Generation	27
4.4	AI Agent Workflow Implementation	28
4.4.1	Shared State	28
4.4.2	Processing Nodes	28
4.4.3	Workflow Execution	28
4.5	Testing and Validation	29
4.5.1	Individual Tool Testing	29

4.5.2	End-to-End Workflow Testing	30
4.5.3	Result Validation	30
4.6	Challenges Encountered and Solutions	30
4.7	Chapter Summary	31
5	Conclusion and Future Work	32
5.1	Achievements	32
5.2	Skills Acquired	33
5.2.1	Technical Skills	33
5.2.2	Professional Skills	34
5.3	Project Limitations	34
5.4	Future Work	35
5.5	General Conclusion	36

List of Tables

1.1	Main challenges in Automotive Ethernet trace analysis	5
3.1	Main functional requirements	17
3.2	Information maintained during the agent workflow	22

List of Abbreviations

AI	Artificial Intelligence
ARP	Address Resolution Protocol
CI/CD	Continuous Integration and Continuous Deployment
DHCP	Dynamic Host Configuration Protocol
DoIP	Diagnostics over Internet Protocol
IP	Internet Protocol
LLM	Large Language Model
PCAP	Packet Capture
SOME/IP	Scalable service-Oriented MiddlewarE over IP
TCP	Transmission Control Protocol
UDP	User Datagram Protocol

General Introduction

The automotive industry is undergoing a significant technological transformation driven by the increasing integration of electronics, software, connectivity, and advanced communication systems into modern vehicles. As vehicle architectures become more complex, the volume of data exchanged between their different electronic components continues to increase. This evolution has created new communication requirements and has reinforced the importance of high-performance networking technologies, particularly in the context of Automotive Ethernet.

The growing complexity of automotive communication systems also creates important challenges for testing and validation activities. During these activities, network traffic can be captured and stored in the form of Packet Capture (PCAP) traces. These traces contain valuable information about the communications occurring within a network and can be used to investigate system behavior, protocol exchanges, and potential communication issues. However, a single test scenario may generate a large volume of captured data, making manual analysis increasingly time-consuming and difficult.

Moreover, captured network traces may contain repetitive, redundant, or low-value traffic in addition to information that is relevant to the analysis objective. Consequently, engineers may need to spend considerable time filtering and organizing captured communications before meaningful analysis can be performed. This highlights the importance of an efficient preprocessing stage capable of reducing unnecessary information while preserving the relevant data required for subsequent processing.

At the same time, the development of artificial intelligence techniques creates new opportunities for supporting network analysis and testing activities. AI-based systems can assist in the processing of large volumes of communication data and support tasks such as pattern recognition, event identification, traffic analysis, and anomaly detection. However, the effectiveness of such approaches depends strongly on the quality, relevance, and structure of the information provided to them. Preparing network traces in a consistent and lightweight form therefore represents an important prerequisite for efficient automated analysis.

It is within this context that the present internship project, entitled “*PCAP Preprocessing AI Agent for Automotive Ethernet Testing*”, was carried out. The objective of the project is to develop an automated preprocessing approach for Automotive Ethernet communication traces. The proposed solution aims to process captured communication data, extract relevant information, filter unnecessary or redundant traffic, and generate structured outputs suitable for downstream processing and AI-based analysis.

This work is conducted within an industrial context related to automotive testing and validation. By automating part of the trace preparation process, the proposed approach seeks to reduce the amount of information that must be manually investigated and to provide a clearer and more structured communication context for subsequent analysis. The resulting preprocessing pipeline is intended to contribute to more efficient workflows for the investigation and preparation of network communication traces.

The remainder of this report is organized into five chapters. Chapter 1 presents the project context, including the host organization, the challenges associated with Automotive Ethernet testing, the identified limitations, and the proposed solution. Chapter 2 introduces the main technologies and tools relevant to the project, with particular attention to Automotive Ethernet, automotive communication protocols, and the technologies used for data processing and AI agent development. Chapter 3 presents the requirements and the overall architecture of the proposed system. Chapter 4 details the implementation of the solution, together with the testing, validation, and obtained results. Finally, Chapter 5 concludes the report by summarizing the main achievements, discussing the limitations of the work, and presenting possible directions for future improvements.

CHAPTER 1

Project Context

1.1 Host Organization Presentation

1.1.1 Company Overview

The internship was carried out within KPIT Technologies, a company specializing in software and engineering solutions for the automotive and mobility industry. KPIT focuses on supporting the transformation of the automotive sector toward increasingly software-driven and intelligent vehicles.

With more than twenty-five years of experience in the mobility domain, KPIT develops and integrates software solutions for different areas of vehicle development. Its activities cover a broad range of automotive engineering domains, including autonomous driving and Advanced Driver Assistance Systems (ADAS), body electronics, chassis, cockpit systems, propulsion, electrical and electronic architectures, middleware, connected mobility, cloud and data-related solutions, and vehicle engineering.

The company's activities also include automotive software and systems engineering services such as embedded software development, systems integration, testing and validation, simulation, continuous integration and continuous delivery, as well as functional safety and compliance. These activities reflect the increasing importance of software throughout the lifecycle of modern vehicles.

KPIT operates through a global engineering network and collaborates with automotive manufacturers and mobility partners across different regions. The company's engineering activities are organized around the development, integration, testing, and validation of technologies that contribute to the evolution toward Software-Defined Vehicles (SDVs).

1.1.2 KPIT Tunisia and Organizational Context

KPIT has established an engineering center in Tunisia as part of its global delivery network. The Tunisia center contributes to international automotive engineering programs and works

in collaboration with engineering teams located in other regions, including Europe, Asia, and the Americas.

The presence of KPIT in Tunisia provides access to local engineering talent while enabling close collaboration with international teams and automotive projects. The center contributes to several areas of expertise, including electrification, body electronics, virtual engineering, connected mobility and digital cockpit, autonomous driving and ADAS, and electrical and electronic middleware and architecture.

Rather than operating independently, the Tunisia engineering center is integrated into KPIT's wider engineering organization. This international structure enables engineering teams to collaborate on complex automotive programs and to contribute to projects involving different technologies and areas of expertise.

The present internship was conducted within this automotive engineering environment and is associated with activities related to the analysis and preparation of communication data generated during Automotive Ethernet testing and validation processes.

1.1.3 Testing and Validation Context

Testing and validation represent important activities within automotive software and systems engineering. As vehicle architectures become increasingly software-intensive and interconnected, verifying the correct behavior of individual components and their interactions becomes essential throughout the development process.

KPIT provides testing and validation capabilities covering several areas of automotive development. These include electrical and electronic architecture and network testing, virtual testing, multi-domain testing, vehicle testing, and platform-level validation.

Within this context, network testing plays an important role in verifying communication between automotive systems. Captured network traffic can provide valuable information regarding communication flows, protocol exchanges, timing behavior, and interactions between network participants.

The increasing complexity of automotive communication systems also increases the amount of data generated during testing activities. Consequently, efficient approaches for collecting, processing, filtering, and organizing communication traces are important for supporting testing and validation workflows.

The internship project is positioned within this context. It focuses specifically on the preparation of captured Automotive Ethernet communication data before subsequent processing and analysis.

1.2 Problem Statement

1.2.1 Automotive Ethernet Testing Challenges

Table 1.1 – Main challenges in Automotive Ethernet trace analysis

Challenge	Impact on Analysis
Large communication volume	Increases the amount of data that must be processed and investigated during testing activities.
Multiple communication flows	Makes it more difficult to identify the exchanges relevant to a specific analysis objective.
Protocol diversity	Requires communication data to be interpreted according to different protocol layers and message types.
Repetitive or redundant traffic	Increases dataset size without necessarily providing additional analytical value.
Preservation of communication context	Requires filtering operations to retain the information necessary for accurate downstream analysis.

Automotive Ethernet systems can generate a significant volume of communication data during development and testing activities. A single test scenario may involve multiple communication participants, protocols, services, and simultaneous data flows. Capturing this communication produces PCAP traces containing detailed records of the network traffic observed during the test.

Although these traces represent a valuable source of information, their increasing size and complexity create practical challenges for engineers. Relevant information may be distributed across a large number of packets and communication flows, making the investigation of specific events or behaviors difficult.

In addition, Automotive Ethernet traces may contain different categories of traffic. Depending on the tested system and the communication environment, the captured data may include transport-layer traffic, service-oriented communication, diagnostic exchanges, service discovery messages, and network management traffic. Not all of this information is necessarily relevant to a particular analysis objective.

Another challenge is the temporal nature of network communication. The meaning of an individual packet can depend on the packets that preceded or followed it. Therefore, reducing the amount of captured data must be performed carefully to avoid removing information required to understand the overall communication context.

Table 1.1 summarizes the principal challenges addressed by the project and their impact on network trace analysis.

These challenges demonstrate that the difficulty is not limited to storing captured traffic. The main objective is to transform raw communication data into a representation that is more manageable while preserving the information required for meaningful analysis.

1.2.2 Current Limitations

Traditional PCAP analysis often relies on manual inspection and filtering performed by engineers using network analysis tools. Although such tools provide powerful capabilities, the investigation of large traces can still require significant manual effort.

Engineers may need to identify relevant protocols, apply filtering criteria, inspect communication sequences, and compare packets across different flows. These operations can become time-consuming when the trace contains a large amount of traffic or when several communication behaviors must be investigated.

Another limitation concerns the direct use of raw PCAP data in automated analysis systems. Raw traces are primarily designed to preserve packet-level information and may not provide a compact representation suitable for higher-level processing. Before automated or AI-based analysis can be performed, relevant information often needs to be extracted and organized into a more structured form.

Furthermore, simple filtering based exclusively on predefined rules may not always be sufficient. A packet that appears repetitive in isolation may still contribute to the interpretation of communication timing, sequence, or system behavior. Consequently, the filtering process must balance data reduction with the preservation of relevant context.

The identified problem can therefore be summarized as follows:

How can large and noisy Automotive Ethernet PCAP traces be automatically transformed into a structured and lightweight communication context while preserving the information required for reliable downstream analysis?

Addressing this problem requires a preprocessing approach capable of processing raw communication data, extracting relevant information, reducing unnecessary traffic, and organizing the resulting information for subsequent analysis.

1.3 Proposed Solution

1.3.1 PCAP Preprocessing AI Agent

To address the identified challenges, this project proposes a PCAP preprocessing approach organized around an AI agent architecture. The objective of the proposed solution is to automate the preparation of Automotive Ethernet communication traces before they are provided to downstream analysis processes.

The preprocessing workflow begins with the processing of captured network data. Relevant communication information is extracted from the input traces and organized into a form that can be used by subsequent preprocessing operations.

The extracted information is then processed to reduce unnecessary or repetitive traffic. The objective of this stage is not simply to reduce the volume of data, but to preserve the communication information required to maintain an accurate representation of relevant network behavior.

The resulting information is subsequently organized into a structured communication context. This context is intended to provide downstream analysis systems with relevant metadata, communication information, detected events, and other useful characteristics without requiring direct processing of the complete raw trace.

The proposed architecture therefore establishes a separation between raw network data and higher-level analysis. Rather than providing a downstream analysis agent with an entire PCAP trace, the preprocessing workflow prepares a reduced and structured representation of the communication data.

The project is designed as a reusable preprocessing approach. Its purpose is to support the preparation of network traces that may later be used by different downstream applications, including protocol analysis, automated validation, anomaly investigation, or AI-based analysis.

1.3.2 Expected Benefits

The proposed solution is expected to provide several benefits for Automotive Ethernet testing and validation workflows.

First, automated preprocessing can reduce the amount of manual effort required to prepare large communication traces for investigation. By automating extraction, filtering, and context preparation operations, engineers can spend less time performing repetitive data preparation tasks.

Second, the reduction of unnecessary or redundant information can improve the readability and usability of captured communication data. A smaller and more structured representation can make it easier to identify relevant communication flows and events.

Third, the proposed approach can provide a more consistent representation of network communication data. Standardizing the preprocessing workflow can reduce variations caused by manual preparation and facilitate the use of the resulting information by downstream tools.

Finally, the generated communication context can support future AI-based analysis. By providing relevant and structured input data, the preprocessing workflow can establish a foundation for more advanced capabilities such as communication pattern analysis, anomaly detection, automated event investigation, and intelligent trace analysis.

Overall, the proposed solution aims to establish a reusable preprocessing stage between raw Automotive Ethernet packet captures and higher-level analysis. Its purpose is to reduce data complexity while preserving meaningful communication information, thereby supporting more efficient and scalable testing and validation workflows.

CHAPTER 2

Technologies and Tools

This chapter presents the main technologies and tools relevant to the development of the proposed PCAP preprocessing system. The chapter first introduces the communication technologies used in Automotive Ethernet environments, followed by the principal automotive communication protocols that may be encountered in captured network traces. The data processing and AI agent technologies used to develop the solution are then presented, together with the main elements of the development environment.

2.1 Automotive Ethernet Technologies

2.1.1 Ethernet Fundamentals

Ethernet is one of the most widely used networking technologies for data communication. It defines mechanisms for the transmission of data between network devices and provides the foundation for communication in many computer and industrial networks.

At the data link layer, Ethernet communication is based on frames. An Ethernet frame contains addressing information, control fields, and the transmitted data. The source and destination devices are identified through Media Access Control (MAC) addresses, allowing frames to be delivered within a local network.

The increasing communication requirements of modern vehicles have made Ethernet particularly relevant to the automotive domain. Compared with traditional automotive communication technologies designed for lower data rates and specific applications, Ethernet provides higher bandwidth and supports the integration of widely used networking technologies.

For the purposes of this project, Ethernet represents the fundamental communication layer from which captured packet information is processed. Ethernet frames provide important information for network trace analysis, including source and destination addresses, protocol identifiers, and the encapsulated communication data.

2.1.2 Automotive Ethernet

Automotive Ethernet refers to the use and adaptation of Ethernet technologies for communication within vehicles. It has become increasingly important as modern automotive systems require higher bandwidth and more flexible communication architectures.

Automotive applications such as advanced driver assistance systems, cameras, infotainment platforms, connected services, and high-performance computing systems can generate large volumes of communication data. Automotive Ethernet provides a suitable communication infrastructure for supporting these requirements.

Automotive Ethernet environments may contain multiple communication participants and simultaneous data flows. Different electronic systems can exchange information using standard Internet protocols as well as automotive-specific protocols.

From a testing perspective, this communication complexity makes network monitoring and analysis particularly important. Captured Automotive Ethernet traffic may contain packets belonging to different communication flows, protocol layers, and services. The preprocessing of PCAP traces must therefore identify and organize relevant information while maintaining the relationships between communication events.

2.1.3 Virtual Local Area Networks

A Virtual Local Area Network (VLAN) is a mechanism used to logically divide a network into separate communication domains. VLANs can allow different categories of traffic to share the same physical network infrastructure while remaining logically separated.

VLAN information is typically carried within an IEEE 802.1Q tag inserted into an Ethernet frame. The tag can contain information used to identify the VLAN to which a frame belongs.

In Automotive Ethernet systems, VLANs can be used to organize communication and separate traffic associated with different network domains or functions. For PCAP analysis, VLAN information can provide additional context regarding the origin and logical grouping of captured communication.

Therefore, when present in a captured trace, VLAN information can be considered a relevant feature during packet parsing and preprocessing.

2.1.4 TCP, IP, and UDP

The Internet Protocol (IP) provides addressing and routing mechanisms for communication between network participants. Within an IP-based communication architecture, devices are

identified using IP addresses, allowing data to be exchanged between different communication endpoints.

Two commonly used transport protocols operating above IP are the Transmission Control Protocol (TCP) and the User Datagram Protocol (UDP).

TCP provides connection-oriented communication. It establishes a communication session between endpoints and includes mechanisms intended to support reliable and ordered data delivery. TCP is therefore suitable for applications that require reliable communication between participants.

UDP provides connectionless communication with a simpler communication model. It does not establish a connection before transmitting data and does not provide the same reliability mechanisms as TCP. Its reduced communication overhead makes it suitable for applications where low latency and efficient data transmission are important.

Both TCP and UDP can be encountered in Automotive Ethernet communication traces. For this reason, transport-layer information such as source and destination ports, communication endpoints, and protocol type can provide important context during the preprocessing of captured network traffic.

2.2 Automotive Communication Protocols

2.2.1 SOME/IP

Scalable service-Oriented MiddlewarE over IP (SOME/IP) is a communication middleware technology designed for service-oriented communication in automotive systems.

SOME/IP enables communication between software components using services, methods, events, and fields. A service can provide one or more functions that are accessed by other components within the communication system.

In a typical service-oriented communication model, a service provider offers functionality while one or more service consumers access that functionality. SOME/IP can support communication using both TCP and UDP, depending on the requirements of the application.

From a PCAP analysis perspective, SOME/IP messages contain information that can help identify service-oriented communication. Relevant information may include service identifiers, method identifiers, message types, and communication endpoints.

The ability to identify SOME/IP communication is therefore important when processing Automotive Ethernet traces because it allows packet-level information to be associated with higher-level service interactions.

2.2.2 SOME/IP Service Discovery

SOME/IP Service Discovery (SOME/IP-SD) is used to support the discovery and availability management of SOME/IP services.

In a service-oriented architecture, communication participants must be able to identify available services and determine how they can communicate with them. SOME/IP-SD supports this process by allowing services to announce their availability and enabling consumers to discover offered services.

Service Discovery communication can provide useful information about the relationships between communication participants and the services available within a network.

However, such messages may also occur repeatedly during normal system operation. During preprocessing, the frequency and relevance of Service Discovery traffic must therefore be considered carefully. Repetitive communication should not be removed without considering whether it contributes useful information about service availability or changes in system behavior.

2.2.3 Diagnostics over Internet Protocol

Diagnostics over Internet Protocol (DoIP) is an automotive communication protocol designed to support diagnostic communication using IP-based networks.

DoIP enables diagnostic tools and vehicle electronic systems to exchange diagnostic information through Ethernet-based communication. It can therefore provide a high-bandwidth alternative to diagnostic communication mechanisms based on traditional automotive networks.

In Automotive Ethernet traces, DoIP communication may contain information related to diagnostic sessions, vehicle identification, routing activation, and diagnostic message exchanges.

The identification of DoIP traffic is relevant to network preprocessing because diagnostic communication may represent an important part of the captured testing context. Extracting protocol information and communication events associated with DoIP can therefore support subsequent analysis and validation activities.

2.2.4 Address Resolution Protocol and Dynamic Host Configuration Protocol

The Address Resolution Protocol (ARP) is used to associate IP addresses with MAC addresses within a local network. When a device needs to communicate with another participant on the

same network, ARP can be used to determine the corresponding hardware address.

The Dynamic Host Configuration Protocol (DHCP) is used to provide network configuration information dynamically. Depending on the communication environment, DHCP can provide devices with configuration information such as IP addresses and other network parameters.

ARP and DHCP traffic can provide useful information regarding network configuration and the presence of communication participants. However, depending on the analysis objective, some management-related traffic may provide limited value for downstream analysis.

The preprocessing process must therefore distinguish between traffic that is relevant for understanding the communication context and traffic that can be reduced or summarized without affecting the usefulness of the resulting dataset.

2.3 Data Processing and AI Agent Technologies

2.3.1 Python

Python was used as the primary programming language for the development of the project. Its readability, extensive ecosystem, and support for data processing and artificial intelligence applications make it suitable for the development of an automated preprocessing workflow.

Python provides access to libraries for network analysis, data manipulation, and application development. This makes it possible to implement different stages of the processing pipeline within a common programming environment.

Within this project, Python is used to implement the PCAP processing logic, communication information extraction, filtering operations, and the AI agent workflow.

2.3.2 Scapy

Scapy is a Python library used for the manipulation and analysis of network packets. It provides mechanisms for reading packet capture files, inspecting protocol layers, and accessing packet-level information.

For this project, Scapy is used to process captured network traffic and extract relevant communication information from PCAP traces. It allows the preprocessing logic to access different protocol layers and obtain information such as timestamps, MAC addresses, IP addresses, transport protocols, ports, and other available packet attributes.

Scapy therefore represents an important component of the preprocessing stage because it provides the interface between the raw PCAP trace and the structured communication informa-

tion used by subsequent processing operations.

2.3.3 LangChain

LangChain is a framework designed to support the development of applications based on Large Language Models (LLMs). It provides mechanisms for managing interactions between language models, prompts, messages, and external tools.

Within the context of this project, LangChain is used to support the integration of the language model with specialized processing tools. Instead of requiring the language model to directly process the complete network trace, dedicated tools perform specific preprocessing operations and provide the relevant results to the agent workflow.

This separation allows the language model to participate in the coordination and interpretation of processing operations while specialized Python functions handle the underlying data processing tasks.

2.3.4 LangGraph

LangGraph is used to organize the AI agent workflow as a graph-based execution process. The framework makes it possible to define states, processing nodes, transitions, and conditional execution paths.

Within the proposed system, LangGraph is used to structure the interaction between the language model and the available preprocessing tools. The workflow can maintain information through a shared state while processing nodes perform specific operations.

Conditional edges can be used to determine the next step in the workflow based on the result produced by the language model. When a tool call is required, execution can be directed toward the corresponding tool node. Otherwise, the workflow can proceed to completion.

This graph-based approach provides a structured way to organize the agent workflow and supports the integration of multiple preprocessing tools within a common architecture.

2.4 Development Environment

2.4.1 Visual Studio Code

Visual Studio Code was used as the primary development environment for the implementation of the project. It provides support for Python development, code editing, debugging, extension management, and integration with version control systems.

The use of an integrated development environment facilitates the organization of the project and the management of its different source files and components.

2.4.2 Git

Git is a distributed version control system used to manage changes to source code during software development.

Within the project environment, version control supports the tracking of modifications and the management of different versions of the source code. It also facilitates collaboration by providing mechanisms for organizing and sharing changes between developers.

2.5 Chapter Summary

This chapter presented the principal technologies and tools relevant to the development of the PCAP preprocessing system.

The chapter first introduced Automotive Ethernet and the main networking technologies that provide the foundation for captured communication. It then presented several automotive communication protocols, including SOME/IP, SOME/IP Service Discovery, and DoIP, together with network protocols such as ARP and DHCP that may be encountered during trace analysis.

The chapter also presented the main software technologies used to develop the proposed solution. Python and Scapy provide the technical foundation for processing and extracting information from PCAP traces, while LangChain and LangGraph support the implementation and organization of the AI agent workflow. Finally, Visual Studio Code and Git provide the development and source code management environment.

The following chapter presents the requirements and system architecture of the proposed PCAP preprocessing solution, describing how these technologies are combined to support the overall processing workflow.

CHAPTER 3

System Architecture

This chapter presents the requirements analysis and the overall architecture of the proposed PCAP preprocessing system. The objective of the architecture is to transform raw Automotive Ethernet packet capture data into a reduced and structured communication context that can be used by downstream analysis systems.

The proposed solution is organized as an AI agent workflow in which specialized tools perform successive preprocessing operations. The architecture separates the processing logic from the agent orchestration layer, allowing each component to have a clearly defined responsibility.

3.1 Requirements Analysis

3.1.1 Functional Requirements

The main functional requirements of the proposed system were defined according to the objectives of the PCAP preprocessing workflow.

First, the system must be able to receive an input communication trace and make it available for processing. The preprocessing workflow must then extract relevant information from the captured traffic.

The system must also be able to identify meaningful communication events and organize the extracted information in a structured form. This intermediate representation is required to avoid passing the complete raw trace directly to subsequent processing stages.

Another important requirement is the reduction of unnecessary information. The system must identify repetitive, redundant, or low-value communication data and reduce its impact on the final communication context while preserving information required for accurate analysis.

Finally, the system must aggregate the outputs produced during preprocessing and generate a structured final result that can be consumed by a downstream AI analysis system.

The principal functional requirements are summarized in Table 3.1.

Table 3.1 – Main functional requirements

Requirement	Description
Input processing	Receive and process communication data originating from a PCAP trace.
Event extraction	Extract meaningful communication events from the processed trace data.
Noise filtering	Identify and reduce repetitive, redundant, or low-value communication information.
Context preservation	Preserve the information required to maintain an accurate representation of relevant communication behavior.
Context generation	Aggregate the results of the preprocessing operations into a structured communication context.
Agent orchestration	Coordinate the interaction between the language model and the available preprocessing tools.
Output generation	Produce a standardized result suitable for downstream automated or AI-based analysis.

3.1.2 Non-Functional Requirements

In addition to its functional requirements, the system must satisfy several non-functional requirements related to its use as a preprocessing solution.

Maintainability is important because the preprocessing workflow may evolve to support additional protocols, filtering criteria, or analysis capabilities. The architecture should therefore separate the main processing responsibilities and avoid unnecessary dependencies between components.

Scalability is also relevant because PCAP traces can vary considerably in size. The preprocessing approach should reduce the amount of information passed to later stages and avoid unnecessary processing of data that does not contribute useful context.

Accuracy is particularly important during filtering and compression. The objective is not simply to reduce the volume of communication data. The resulting context must preserve the information required for reliable downstream analysis.

Reusability is another important requirement. The generated communication context should be suitable for different downstream applications, including automated investigation, protocol analysis, validation activities, and future AI-based analysis.

Modularity is required to allow the different processing operations to be developed and maintained independently while remaining integrated within the same agent workflow.

3.2 Global Architecture

3.2.1 Architectural Overview

The proposed system follows a modular preprocessing architecture based on an AI agent and a set of specialized processing tools.

The architecture distinguishes between two main responsibilities. The first responsibility is **agent orchestration**, which determines how the workflow is executed and coordinates the interaction with the available tools. The second responsibility is **data processing**, which performs the actual extraction, filtering, compression, and context generation operations.

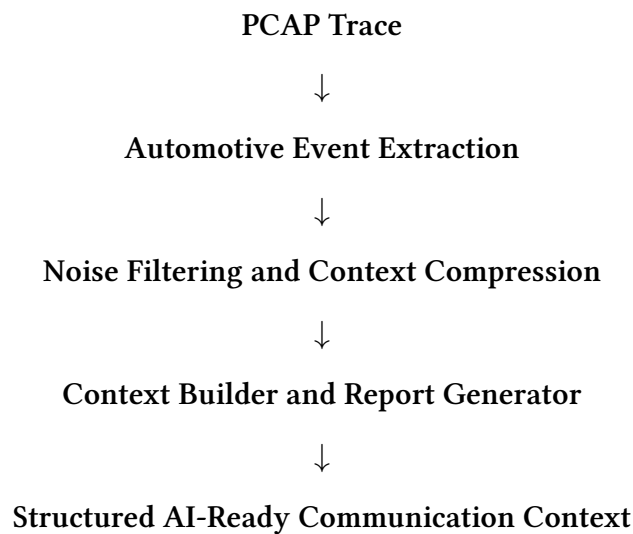
This separation ensures that the agent architecture does not contain unnecessary packet-processing logic. Instead, specialized tools are responsible for performing the corresponding processing operations.

The overall workflow is based on three main preprocessing stages:

1. Automotive event extraction;
2. Noise filtering and context compression;
3. Context building and report generation.

Each stage receives information produced by the previous stage and transforms it into a more suitable representation for subsequent processing.

The resulting architecture can therefore be represented as follows:



This sequential representation describes the logical flow of communication information through the preprocessing pipeline. The internal execution of the system is coordinated through the agent workflow.

3.2.2 Data Input Layer

The data input layer represents the starting point of the preprocessing process. Its purpose is to provide captured network communication data to the processing workflow.

The original source of the communication data is a PCAP trace containing captured network packets. PCAP traces can contain information associated with multiple protocol layers, communication participants, and communication flows.

The raw trace is not intended to be used directly as the final input to the downstream analysis system. Instead, it represents the source from which relevant communication information is progressively extracted and transformed by the preprocessing workflow.

This approach reduces the dependency of later processing stages on the complete raw trace and enables the system to operate on increasingly structured representations of the communication data.

3.2.3 Agent Orchestration Layer

The agent orchestration layer is responsible for organizing the execution of the preprocessing workflow.

The language model acts as the decision-making component of the agent architecture. It can interact with the available tools through the agent workflow and determine whether a specialized processing operation must be executed.

The workflow maintains a shared state containing the information required during execution. Processing nodes receive the current state, perform their respective operations, and return updated information to the workflow.

Conditional transitions are used to determine the next step after the language model produces a response. When the workflow requires the execution of a processing tool, control is directed toward the corresponding tool node. Otherwise, the execution can terminate.

The orchestration layer therefore provides a structured mechanism for coordinating the processing tools without embedding the detailed preprocessing logic directly within the agent definition.

3.3 Preprocessing Tools

3.3.1 Automotive Event Extraction

The first preprocessing stage is responsible for extracting meaningful automotive communication events from the input trace.

The purpose of this stage is to transform low-level captured communication data into a representation that is more relevant to the subsequent processing workflow. Instead of treating the complete trace only as a collection of independent packets, the extraction stage identifies communication information that can contribute to the understanding of system behavior.

The output of this tool serves as the input to the filtering and context compression stage. This intermediate representation contains the extracted information required to determine which communications should be retained, reduced, or summarized.

The event extraction stage therefore establishes the initial transition from raw communication data toward higher-level processing.

3.3.2 Noise Filtering and Context Compression

The second preprocessing stage receives the output produced by the automotive event extraction stage.

Its objective is to reduce the amount of repetitive or low-value information while preserving the communication context required for accurate downstream analysis.

The filtering process may address categories of traffic such as repetitive events, duplicate information, keep-alive communication, broadcasts, and management-related traffic when these communications do not provide additional analytical value for the resulting context.

However, the filtering stage is not intended to remove communication information blindly. Network communication is contextual, and apparently repetitive information can still contribute to the interpretation of timing, sequences, or changes in system behavior.

For this reason, the purpose of context compression is to preserve meaningful information while reducing unnecessary repetition. The resulting output should provide a smaller and more manageable representation than the original extracted communication data without losing the information required by subsequent analysis.

3.3.3 Context Builder and Report Generator

The final preprocessing stage receives the outputs generated by the preceding processing tools.

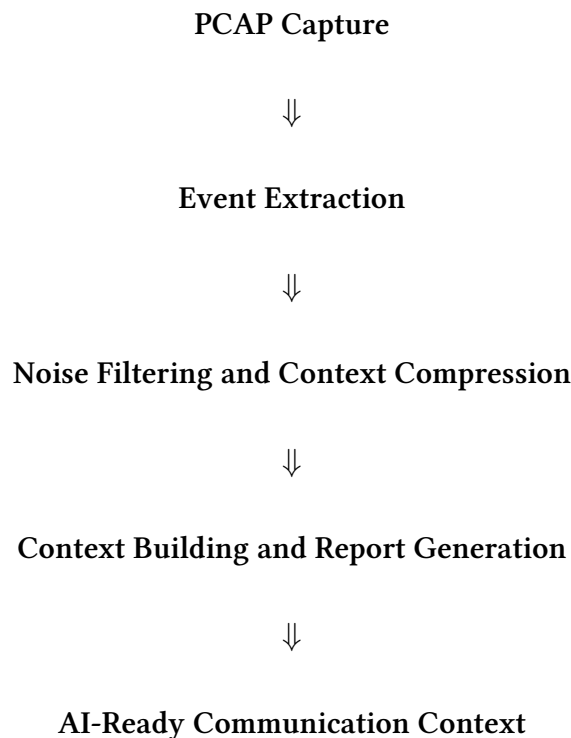
Its purpose is to aggregate the available information into a standardized communication context suitable for downstream AI-based analysis.

The resulting context can include relevant metadata, detected communication information, extracted events, timing-related information, anomalies when available, and summary statistics produced during preprocessing.

The context builder acts as the boundary between the preprocessing workflow and the downstream analysis system. It organizes the information generated by the preceding stages into a consistent output structure, reducing the need for the downstream analysis system to interpret raw PCAP data directly.

This separation also supports future extensions because additional preprocessing information can be incorporated into the final communication context without requiring changes to the fundamental architecture of the downstream analysis system.

3.4 Data Processing Workflow



The data processing workflow describes how information is progressively transformed from raw captured traffic into a structured communication context.

The process begins with the original PCAP trace. The trace contains the raw network communication data captured during the testing environment.

During the first stage, relevant automotive communication events are extracted from the avail-

able trace information. This produces an intermediate representation focused on meaningful communication activity.

The extracted information is then provided to the filtering and context compression stage. At this point, repetitive or low-value information is reduced while relevant communication context is preserved.

Finally, the remaining information is aggregated by the context builder and report generator. The result is a structured output containing the relevant information required by downstream analysis.

The complete logical workflow can therefore be summarized as follows:

An important characteristic of this workflow is that each stage operates on the result produced by the preceding stage. The preprocessing process therefore progressively reduces the dependency on the complete raw trace and produces increasingly structured information.

3.5 State and Information Flow

Table 3.2 – Information maintained during the agent workflow

Information	Purpose
Processing request	Defines the requested preprocessing operation or analysis objective.
Agent messages	Maintains the interaction between the language model and the workflow.
Input information	Identifies the communication trace or data provided to the preprocessing process.
Tool outputs	Stores the results produced by the specialized preprocessing operations.
Generated context	Contains the structured information prepared for downstream analysis.

The proposed architecture uses a shared state to maintain information during the execution of the agent workflow.

The state represents the communication mechanism between the different processing nodes. Rather than passing information through unrelated global variables, the workflow maintains the relevant execution data in a common structure.

The information stored during execution can include the user request or processing prompt,

the messages exchanged with the language model, the input trace information, and the outputs generated by the preprocessing tools.

Each node receives the current state as input and returns the information generated during its execution. This approach allows the workflow to preserve relevant information while progressing through the different processing stages.

The following table summarizes the main categories of information that can be maintained during the workflow.

3.6 Chapter Summary

This chapter presented the requirements and overall architecture of the proposed PCAP preprocessing system.

The architecture is designed as a modular workflow that progressively transforms raw Automotive Ethernet communication data into a structured and reduced communication context. The solution separates agent orchestration from the underlying preprocessing logic and organizes the processing workflow around three main stages: automotive event extraction, noise filtering and context compression, and context building and report generation.

The shared workflow state enables information to be maintained between processing operations, while the graph-based agent architecture provides a structured mechanism for coordinating the language model and the available tools.

The resulting architecture establishes a clear separation between raw captured network data and downstream analysis. By extracting relevant communication information, reducing unnecessary data, and generating a structured context, the proposed solution prepares Automotive Ethernet traces for subsequent automated and AI-based analysis.

The following chapter presents the implementation of the proposed system, including the development of the preprocessing tools, the agent workflow, and the validation of the implemented solution.

CHAPTER 4

Implementation and Validation

This chapter presents the implementation of the proposed PCAP preprocessing system. While the previous chapter described the overall architecture and information flow, this chapter focuses on the practical development of the different software components.

The implementation follows a modular approach in which the agent orchestration, specialized tools, and underlying processing logic are separated according to their responsibilities. The system is organized around three main preprocessing operations: automotive event extraction, noise filtering and context compression, and context building and report generation.

The implementation also includes the agent workflow responsible for coordinating these operations and maintaining the information required during execution.

4.1 Development Methodology

The development of the system followed an incremental implementation approach. The preprocessing workflow was divided into smaller functional components, allowing each processing stage to be developed and validated independently before being integrated into the complete agent workflow.

The development process began with the implementation of the core processing capabilities required to extract relevant information from communication data. The filtering and context compression functionality was then introduced to reduce repetitive or low-value information while preserving meaningful communication context.

Finally, the outputs generated by the preprocessing stages were aggregated into a unified report intended to provide a structured input for downstream AI analysis.

This incremental approach simplified the development and validation process. Each component could be tested individually, making it easier to identify implementation issues before executing the complete workflow.

4.2 Project Implementation Structure

4.2.1 Separation of Responsibilities

The implementation was organized according to a separation between three main responsibilities:

- agent and workflow orchestration;
- specialized preprocessing tools;
- core data processing logic.

This organization was adopted to maintain a clear distinction between workflow coordination and the actual processing of communication data.

The agent layer is responsible for coordinating the execution of the workflow. The tool layer provides the operations that can be invoked by the agent. The core processing layer contains the implementation responsible for performing the underlying extraction, filtering, compression, and report generation operations.

This separation improves maintainability because modifications to the processing logic can be performed without unnecessarily modifying the agent workflow.

4.2.2 Core Processing Layer

The core processing layer contains the main implementation logic of the preprocessing system.

This layer is responsible for performing the actual data transformations required by the different stages of the pipeline. These operations include processing extracted communication information, identifying repetitive or low-value data, preserving relevant context, and preparing the information required for the final report.

Keeping this logic within a dedicated processing layer avoids placing complex processing operations directly inside the agent workflow or tool definitions.

The core processing layer therefore represents the main functional implementation of the preprocessing system.

4.2.3 Tool Layer

The tool layer provides the interface between the agent workflow and the underlying processing operations.

Each tool corresponds to a specific preprocessing capability. Rather than containing unnecessary orchestration logic, the tools provide a clear entry point for invoking the corresponding processing functionality.

The implemented preprocessing tools are:

1. Automotive Event Extraction;
2. Noise Filtering and Context Compression;
3. Context Builder and Report Generation.

The output produced by one preprocessing stage can be used as the input of the next stage, allowing the communication information to be progressively transformed throughout the workflow.

4.2.4 Agent and Workflow Layer

The agent and workflow layer is responsible for coordinating the execution of the preprocessing system.

The workflow defines the processing sequence and manages the interaction between the agent and the available tools. It also maintains the execution state required to preserve information throughout the different stages.

The workflow architecture is designed to avoid embedding detailed packet-processing operations within the agent itself. Instead, specialized tools perform the required preprocessing operations while the workflow coordinates their execution.

4.3 Preprocessing Tool Implementation

4.3.1 Automotive Event Extraction

The first preprocessing tool is responsible for extracting meaningful communication events from the input data.

Its purpose is to provide an initial structured representation of the communication activity that can be processed by subsequent stages. Instead of requiring later stages to operate directly on the complete raw communication trace, the extraction stage identifies information relevant to the preprocessing workflow.

The extracted output acts as the input of the noise filtering and context compression stage.

This separation is important because the second tool does not need to perform the initial event extraction again. Its responsibility is to process and refine the communication information already produced by the first stage.

4.3.2 Noise Filtering and Context Compression

The second tool receives the output generated by the automotive event extraction stage.

The objective of this component is to reduce unnecessary communication information while preserving the information required to maintain an accurate representation of relevant network behavior.

The implementation focuses on identifying information that provides limited additional analytical value when repeated. Depending on the available communication data, this can include repetitive events, duplicate information, keep-alive communication, broadcast traffic, or management-related traffic.

However, the filtering operation is not implemented as a simple process of removing all repeated communication. Repetition can sometimes provide useful information regarding communication behavior, timing, or changes in the state of a network interaction.

For this reason, the filtering process is designed to reduce unnecessary repetition while preserving the communication context required by the next stage of the preprocessing workflow.

The resulting output represents a reduced and more manageable communication context.

4.3.3 Context Builder and Report Generation

The third preprocessing tool is responsible for building the final communication context.

This component receives the information generated by the previous preprocessing stages and aggregates it into a standardized output.

The report generation process organizes the available information into a consistent structure that can be consumed by a downstream analysis system. The generated context can include relevant communication metadata, extracted events, information related to communication flows, detected protocol information, timing-related information, anomalies when available, and summary statistics.

The main purpose of this component is to establish a clear boundary between the preprocessing workflow and the future AI analysis layer.

As a result, the downstream analysis system does not need to directly process the complete raw PCAP trace. Instead, it receives the structured communication context generated by the

preprocessing pipeline.

4.4 AI Agent Workflow Implementation

4.4.1 Shared State

The workflow uses a shared state to maintain information during execution.

The state acts as a common structure through which the different processing nodes access and update the information required by the workflow.

The stored information can include the processing request, agent messages, input information, intermediate tool outputs, and the final generated communication context.

Using a shared state provides a structured mechanism for exchanging information between processing stages. Each node receives the current state, performs its corresponding operation, and returns the updated information.

This approach avoids the use of unrelated global variables and allows the workflow to maintain a clear information flow.

4.4.2 Processing Nodes

The agent workflow is composed of nodes representing the main execution stages.

Each node receives the current workflow state and performs a specific responsibility. Depending on the node, this can include interacting with the language model, invoking a preprocessing tool, or updating the information maintained by the workflow.

The use of dedicated nodes provides a modular representation of the execution process. Individual components can be modified or extended without requiring changes to the complete system.

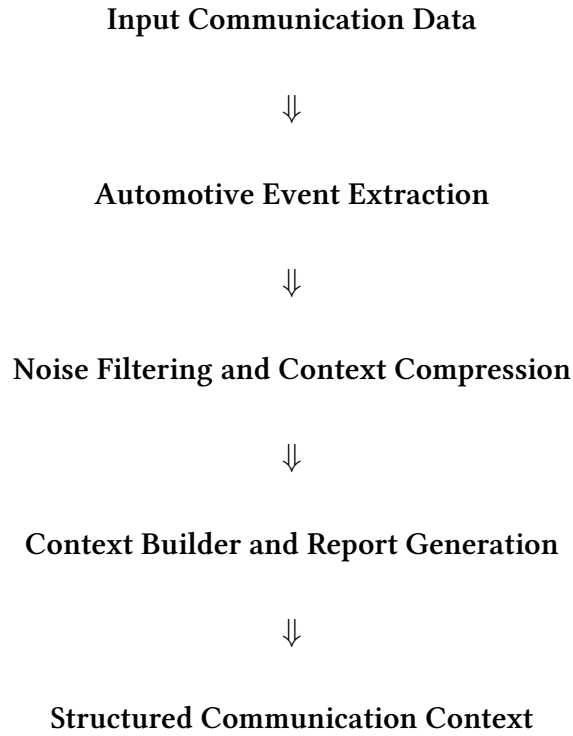
The specialized preprocessing tools are integrated into the workflow through their corresponding processing nodes.

4.4.3 Workflow Execution

The execution of the workflow begins when the input communication data and processing request are provided to the system.

The agent workflow coordinates the available processing operations and maintains the information generated during execution.

The logical preprocessing sequence follows the progression:



The final result represents the output of the complete preprocessing workflow and is intended to be used by a downstream AI analysis component.

4.5 Testing and Validation

4.5.1 Individual Tool Testing

The preprocessing tools were validated individually to verify that each component could execute its intended operation.

Testing the components independently allowed the outputs of each preprocessing stage to be inspected before integration into the complete workflow.

The first tool was validated by verifying that relevant communication events could be extracted from the input data.

The filtering and context compression tool was then validated using the extracted output. The objective of this validation was to verify that unnecessary or repetitive information could be reduced while preserving meaningful communication information.

Finally, the context builder and report generator was tested to verify that the available preprocessing outputs could be aggregated into a consistent final result.

4.5.2 End-to-End Workflow Testing

After the individual preprocessing stages were validated, the complete workflow was executed as an integrated processing pipeline.

This validation focused on verifying the information flow between the different components.

In particular, the output of the automotive event extraction stage was provided to the filtering stage, and the resulting filtered context was subsequently used by the context builder.

This test confirmed that the preprocessing stages could operate together as a complete workflow.

4.5.3 Result Validation

The generated results were inspected to verify their consistency and usefulness for downstream processing.

The validation focused on ensuring that:

- relevant communication information was retained;
- unnecessary or repetitive information was reduced;
- intermediate outputs could be transferred between processing stages;
- the final output provided a structured communication context.

The validation process demonstrated the importance of evaluating not only the execution of individual components but also the consistency of the information passed between the different stages of the workflow.

4.6 Challenges Encountered and Solutions

Several challenges were encountered during the development of the preprocessing workflow.

One of the main challenges was defining a clear separation between the responsibilities of the different components. Early design decisions could lead to processing logic being distributed between the agent workflow and the tool definitions.

This issue was addressed by simplifying the implementation structure. The specialized tools were maintained as the interface for the agent workflow, while the main processing logic was organized within the core processing layer.

Another challenge concerned the filtering stage. The project objective required unnecessary and redundant traffic to be reduced, but the filtering process also needed to preserve information required for downstream analysis.

This was addressed by treating filtering as a context-preserving operation rather than a simple packet removal mechanism. The objective of the stage is therefore to reduce unnecessary information without blindly discarding communication data that may contribute to the interpretation of system behavior.

A further challenge was ensuring that the output of each tool could be used effectively by the following stage. The workflow was therefore organized around a progressive information transformation process in which each preprocessing operation receives information generated by the previous stage.

Finally, the use of multiple software components required the architecture to remain understandable and maintainable. The resulting implementation was simplified by clearly separating the core processing logic, tool definitions, and agent workflow.

4.7 Chapter Summary

This chapter presented the implementation and validation of the proposed PCAP preprocessing system.

The implementation follows a modular organization that separates the agent workflow, specialized preprocessing tools, and underlying processing logic. The system performs three main preprocessing operations: automotive event extraction, noise filtering and context compression, and context building and report generation.

The workflow uses a shared state to maintain information during execution and coordinates the different processing stages through dedicated nodes.

The implementation was validated through individual tool testing and integrated end-to-end workflow testing. These validation activities focused on verifying the correct transformation and transfer of information between the different preprocessing stages.

The implemented system establishes a preprocessing pipeline capable of progressively transforming communication information into a reduced and structured context suitable for downstream AI-based analysis.

CHAPTER 5

Conclusion and Future Work

This chapter presents the overall conclusions of the internship project, summarizes the main achievements, discusses the skills acquired during the development process, identifies the principal limitations of the proposed solution, and presents possible directions for future work.

5.1 Achievements

The objective of this internship project was to develop a preprocessing approach for Automotive Ethernet packet capture data. The proposed solution was designed to transform raw and potentially large PCAP traces into a more structured and lightweight communication context suitable for downstream processing and analysis.

To achieve this objective, the preprocessing workflow was organized around a modular architecture composed of specialized processing stages. The first stage focuses on the extraction of meaningful automotive communication events from the available input data. The second stage performs noise filtering and context compression in order to reduce repetitive or low-value information while preserving the communication context required for subsequent analysis. The final stage aggregates the resulting information and prepares a structured communication context for downstream AI-based processing.

The implementation also introduced an agent-based workflow to coordinate the different preprocessing operations. By separating the orchestration layer, tool definitions, and core processing logic, the resulting architecture provides a clearer organization of responsibilities and supports the independent evolution of the different components.

The main achievements of the project can therefore be summarized as follows:

- the design of a modular preprocessing architecture for Automotive Ethernet communication traces;
- the development of specialized processing stages for event extraction, noise filtering and context compression, and context generation;

- the separation of workflow orchestration from the underlying communication processing logic;
- the progressive transformation of raw communication information into a structured context;
- the implementation of a shared workflow state for maintaining information during execution;
- the preparation of an output intended to support downstream automated and AI-based analysis.

Overall, the project demonstrates the potential of combining traditional network data pre-processing techniques with an AI agent architecture to support the preparation of complex communication traces.

5.2 Skills Acquired

The internship provided an opportunity to develop both technical and professional skills.

5.2.1 Technical Skills

From a technical perspective, the project contributed to a better understanding of network communication and Automotive Ethernet environments. Working with PCAP traces provided practical experience with captured packet data and the challenges associated with extracting useful information from large and heterogeneous communication datasets.

The project also provided experience with network packet processing using Python and Scapy. These technologies were used to support the extraction and processing of communication information from captured network data.

Another important area of learning concerned AI agent development. The use of LangChain and LangGraph introduced concepts related to tool-based agent architectures, shared workflow states, processing nodes, and graph-based execution.

The development process also reinforced software engineering principles related to modularity and separation of responsibilities. Organizing the implementation into distinct layers for processing logic, tools, and workflow orchestration provided practical experience in designing maintainable software architectures.

5.2.2 Professional Skills

In addition to technical knowledge, the internship contributed to the development of several professional skills.

The project required the analysis of technical requirements and the translation of high-level objectives into a concrete software architecture. This process contributed to the development of problem-solving and analytical skills.

The iterative development process also required the identification and resolution of technical challenges. Refining the architecture, clarifying component responsibilities, and validating the information flow between processing stages provided practical experience in approaching complex technical problems incrementally.

Furthermore, the internship environment provided experience with professional software development practices, including the organization of source code, the use of development tools, version control, technical documentation, and communication within an engineering environment.

5.3 Project Limitations

Although the proposed solution establishes a foundation for the preprocessing of Automotive Ethernet communication traces, several limitations remain.

One limitation concerns protocol coverage. Automotive Ethernet environments can include a wide range of protocols and communication behaviors. The current preprocessing approach provides a foundation for handling relevant communication information, but additional protocol-specific processing capabilities may be required to support a wider range of testing scenarios.

Another limitation concerns the filtering process. Determining whether communication information is unnecessary or redundant can depend on the objective of the downstream analysis. Information that provides limited value in one scenario may be relevant in another. Consequently, filtering criteria may need to be adapted according to the intended use of the resulting communication context.

The performance of the preprocessing workflow is also an important consideration. Large PCAP traces may require additional optimization to support efficient processing in production-scale environments.

Finally, the current project focuses primarily on the preprocessing and preparation of communication data. The downstream AI analysis capabilities that can operate on the generated context represent a separate area that can be further developed.

These limitations do not reduce the value of the proposed approach. Instead, they identify opportunities for extending and improving the system as future requirements become more clearly defined.

5.4 Future Work

The proposed system can be extended in several directions.

A first possible improvement would be the expansion of protocol support. Additional automotive and network communication protocols could be integrated into the preprocessing pipeline, allowing the system to provide a richer representation of complex Automotive Ethernet environments.

The filtering and context compression stage could also be further improved. More advanced techniques could be used to adapt filtering decisions according to communication patterns, protocol characteristics, and the requirements of a specific analysis objective.

Another potential direction is the development of advanced AI-based analysis capabilities using the generated communication context. These capabilities could include communication pattern recognition, anomaly detection, automated event investigation, and the identification of unusual network behavior.

The system could also be extended to support more detailed communication flow analysis. Grouping related packets and events into higher-level sessions or interactions could provide additional context for downstream analysis.

Future work could additionally investigate real-time or near-real-time processing. Rather than operating exclusively on previously captured PCAP files, the preprocessing approach could potentially be adapted to process network communication as it is captured during testing activities.

The integration of the solution into automated testing environments represents another possible direction. The preprocessing pipeline could be connected to existing validation workflows, enabling communication traces to be automatically processed following the execution of test scenarios.

Finally, the generated communication context could serve as the foundation for an LLM-powered analysis agent capable of receiving natural language questions from engineers and investigating the corresponding communication data. Such an approach could support more efficient trace investigation by combining automated preprocessing with higher-level AI reasoning.

5.5 General Conclusion

The increasing complexity of modern automotive communication systems creates significant challenges for the analysis of captured network traffic. Large PCAP traces can contain substantial amounts of heterogeneous, repetitive, and low-level communication data, making manual investigation time-consuming and difficult.

This internship project addressed this challenge by proposing and implementing a modular preprocessing approach for Automotive Ethernet communication traces. The solution progressively transforms communication information through event extraction, noise filtering and context compression, and final context generation.

The resulting architecture establishes a separation between raw captured communication data and downstream analysis. Rather than requiring an analysis system to process a complete PCAP trace directly, the preprocessing workflow prepares a reduced and structured communication context containing the information required for subsequent processing.

The project also demonstrated the potential of combining network data processing with an AI agent architecture. The use of specialized tools and a structured workflow provides a flexible foundation that can be extended as additional preprocessing or analysis capabilities are introduced.

In conclusion, the internship achieved its main objective of establishing a reusable preprocessing pipeline for preparing Automotive Ethernet communication data. The proposed approach provides a foundation for future developments in automated trace analysis, intelligent testing, anomaly detection, and AI-assisted investigation within automotive network validation environments.

References

1. Python Software Foundation. *Python Documentation*. Available at: <https://docs.python.org/3/>
2. Scapy Project. *Scapy Documentation*. Available at: <https://scapy.readthedocs.io/>
3. Wireshark Foundation. *Wireshark User's Guide*. Available at: <https://www.wireshark.org/docs/>
4. Internet Engineering Task Force. *Transmission Control Protocol (RFC 9293)*. Available at: <https://www.rfc-editor.org/rfc/rfc9293>
5. Internet Engineering Task Force. *User Datagram Protocol (RFC 768)*. Available at: <https://www.rfc-editor.org/rfc/rfc768>
6. Internet Engineering Task Force. *An Ethernet Address Resolution Protocol (RFC 826)*. Available at: <https://www.rfc-editor.org/rfc/rfc826>
7. Internet Engineering Task Force. *Dynamic Host Configuration Protocol (RFC 2131)*. Available at: <https://www.rfc-editor.org/rfc/rfc2131>
8. AUTOSAR. *Specification of SOME/IP Protocol*. Available at: <https://www.autosar.org/>
9. International Organization for Standardization. *Road Vehicles — Diagnostic Communication over Internet Protocol (ISO 13400)*.
10. LangChain. *LangChain Documentation*. Available at: <https://python.langchain.com/docs/>
11. LangChain. *LangGraph Documentation*. Available at: <https://langchain-ai.github.io/langgraph/>
12. KPIT Technologies. *KPIT Technologies Tunisia*. Available at: <https://www.kpit.com/kpit-tunisia/>